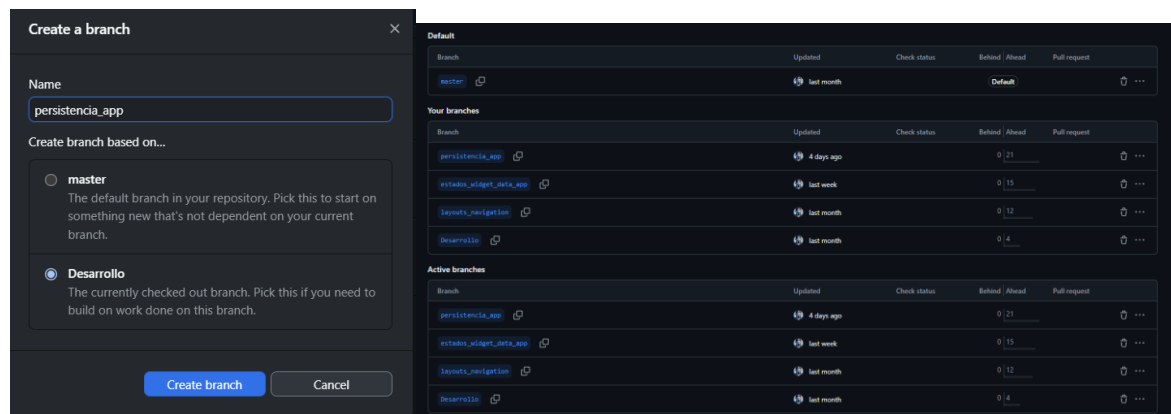


Laboratorio 7

Repositorio: https://github.com/Pasita24/3479A321_2022479621

Actividad 1 y 2

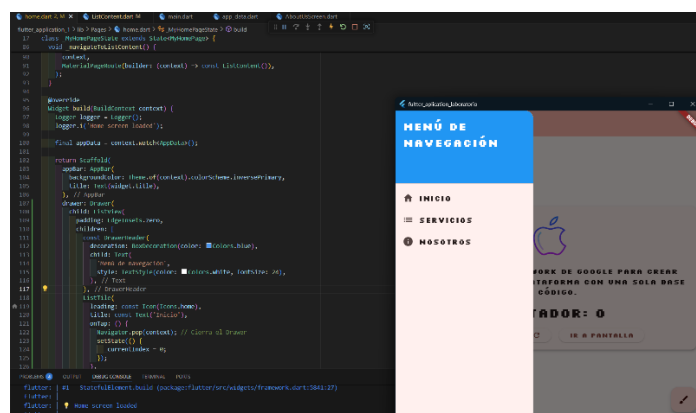
Se realizo la creación de una nueva Branch y migración desde la rama de desarrollo a través de GitHub desktop.



Actividad N°3

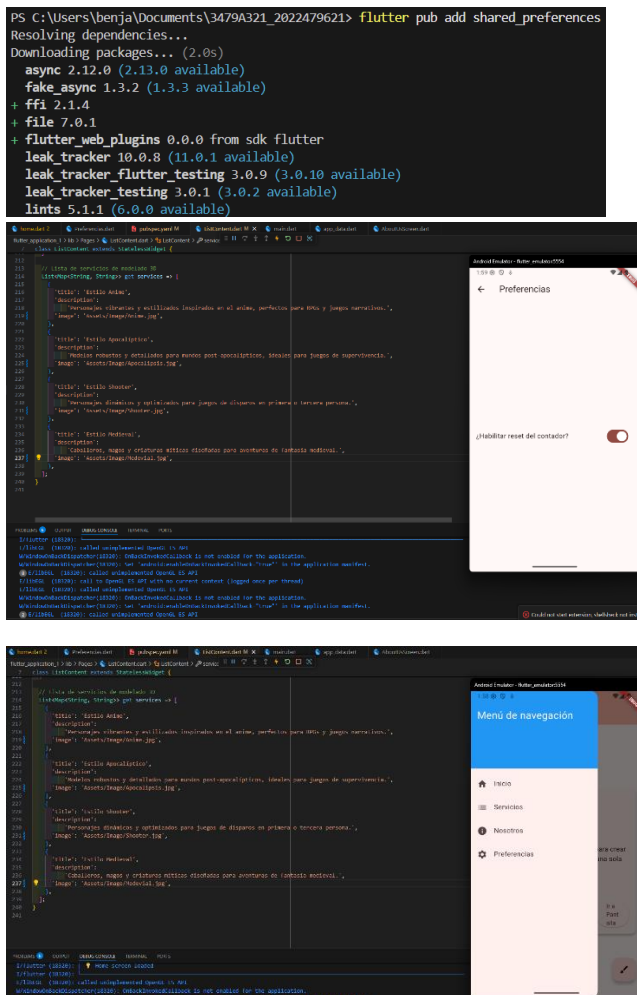
Se elimino los mensajes asociados al método de createState e incluimos el widget de navegación entre las pantallas, para este utilizamos la opción a.- Drawer

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key, required this.title});  
  
  final String title;  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```



Actividad N°4

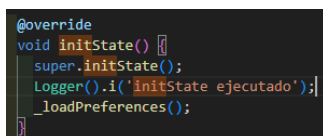
Se instalo el package de shared_preferences a través de la terminal, para posteriormente crear un pantalla llamada Preferencias, esta gestiona el valor de booleano que permite resetear el contador o no.



Actividad N°5

Ahora se modifico la clase de home para poder utilizar el valor de preferencia y estos valores que se producen en la clase home sean actualizados en la pantalla de preferencias

A.-



b.-

```
child: ElevatedButton(
  onPressed: () async {
    final prefs = await SharedPreferences.getInstance();
    final isResetEnabled =
      prefs.getBool('isResetEnabled') ?? false;

    if (isResetEnabled) {
      appData.reset();
      await dbHelper.insertActivity(
        Actividad(
          fecha: DateTime.now(),
          nombre:
            'Reinició contador a ${appData.counter}',
        ), // Actividad
      );
      logger.i('Actividad registrada: Reinició contador');
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('El reinicio está deshabilitado'),
        ), // SnackBar
      );
    }
  },
),
```

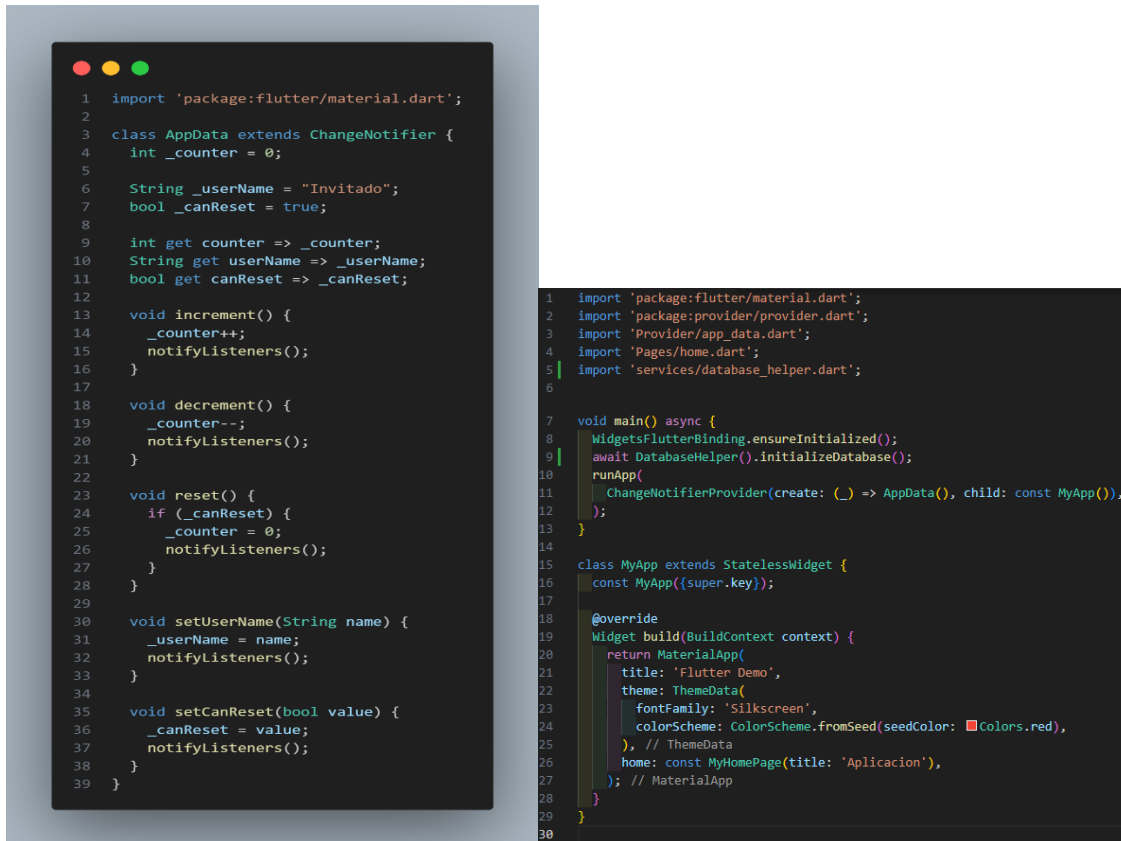
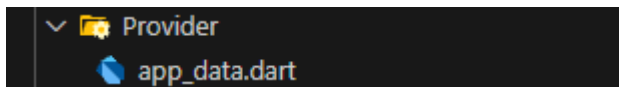
c.-

```
), // ListTile
ListTile(
  leading: const Icon(Icons.settings),
  title: const Text('Preferencias'),
  onTap: () {
    Navigator.pop(context);
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const PreferenciaScreen(),
      ), // MaterialPageRoute
    ).then((_) {
      _loadPreferences();
    });
  },
), // ListTile
```

Actividad N°6

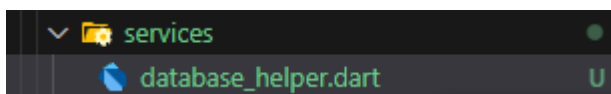
Ahora agregamos la dependencias del package provider, para posteriormente crear la clase actividad, para manejar 3 campos los cuales serán las id, la fecha y la acción correspondiente. También modificaremos la clase main.dart para que pueda trabajar con los servicios de la base de datos.

```
P5 C:\Users\benja\Documents\3479A321_2022479621\flutter_application 1> flutter pub add sqflite path
"sqflite" is already in "dependencies". Will try to update the constraint.
"path" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
async 2.12.0 (2.13.0 available)
fake_async 1.3.2 (1.3.3 available)
flutter_svg 2.0.17 (2.1.0 available)
http 1.3.0 (1.4.0 available)
leak_tracker 10.0.8 (11.0.1 available)
leak_tracker_flutter_testing 3.0.9 (3.0.10 available)
leak_tracker_testing 3.0.1 (3.0.2 available)
lints 5.1.1 (6.0.0 available)
material_color_utilities 0.11.1 (0.12.0 available)
meta 1.16.0 (1.17.0 available)
test_api 0.7.4 (0.7.6 available)
vector_graphics_compiler 1.1.16 (1.1.17 available)
vector_math 2.1.4 (2.2.0 available)
vm_service 14.3.1 (15.0.1 available)
Got dependencies!
14 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
```



Actividad N°7

Se creo una nueva carpeta llamada services, dentro de esta carpeta se creo la clase database_helper.dart y esta clase permite la conexión de la base de datos y la creación de esta misma, también contiene funciones para la recuperación de datos e inserción.



```

1 import 'package:sqflite/sqflite.dart';
2 import 'package:path/path.dart';
3 import '../entity/actividad.dart';
4
5 class DatabaseHelper {
6   static final DatabaseHelper _instance = DatabaseHelper._internal();
7   static Database? _database;
8   static const String tableName = 'actividades';
9
10  factory DatabaseHelper() {
11    return _instance;
12  }
13
14  DatabaseHelper._internal();
15
16  Future<void> initializeDatabase() async {
17    await _database;
18  }
19
20  Future<Database> get database async {
21    if (_database != null) return _database!;
22    _database = await _initDatabase();
23    return _database!;
24  }
25
26  Future<Database> _initDatabase() async {
27    final dbPath = await getDatabasesPath();
28    final path = join(dbPath, 'activity_database.db');
29    return await openDatabase(path, version: 1, onCreate: _onCreate);
30  }
31
32  Future<void> _onCreate(Database db, int version) async {
33    await db.execute('''
34      CREATE TABLE $tableName (
35        id INTEGER PRIMARY KEY AUTOINCREMENT,
36        fecha TEXT NOT NULL,
37        nombre TEXT NOT NULL
38      )
39    ''');
40  }
41
42  Future<void> insertActivity(Actividad actividad) async {
43    final db = await database;
44    await db.insert(
45      tableName,
46      actividad.toMap(),
47      conflictAlgorithm: ConflictAlgorithm.replace,
48    );
49  }
50
51  Future<List<Actividad>> getActividades() async {
52    final db = await database;
53    final maps = await db.query(tableName);
54    return maps.map((map) => Actividad.fromMap(map)).toList();
55  }
56
57  Future<void> updateActivity(Actividad actividad) async {
58    final db = await database;
59    await db.update(
60      tableName,
61      actividad.toMap(),
62      where: 'id = ?',
63      whereArgs: [actividad.id],
64    );
65  }
66
67  Future<void> deleteActivity(int id) async {
68    final db = await database;
69    await db.delete(tableName, where: 'id = ?', whereArgs: [id]);
70  }
71 }
72

```

Actividad N°8

Se creo una nueva pantalla para visualizar las actividades del contenido de la base de datos, para esto modificamos la clase main para trabajar la con la base de datos.

```
Run | Debug | Profile
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await DatabaseHelper().initializeDatabase();
  runApp(
    ChangeNotifierProvider(create: (_) => AppData(), child: const MyApp()),
  );
}
```

The screenshot shows an Android emulator running the application. The screen displays a list of activities under the title 'Actividades Registradas'. The activities are as follows:

Activity	Timestamp	Edit	Delete
Incrementó contador a 1	2025-05-26 05:53:38.149083	✎	🗑
Incrementó contador a 2	2025-05-26 05:53:38.646758	✎	🗑
Incrementó contador a 3	2025-05-26 05:53:45.034561	✎	🗑
Incrementó contador a 4	2025-05-26 05:53:45.197975	✎	🗑
Reinició contador a 0	2025-05-26 05:53:45.483431	✎	🗑
Reinició contador a 0	2025-05-26 05:53:45.881423	✎	🗑
Decrementó contador a -1	2025-05-26 05:53:46.652328	✎	🗑

The code editor on the left shows the implementation of the 'ActividadesScreen' class:

```
1 import 'package:flutter/material.dart';
2 import 'package:logger/logger.dart';
3 import '../entity/actividad.dart';
4 import '../services/database_helper.dart';
5
6 class ActividadesScreen extends StatefulWidget {
7   const ActividadesScreen({super.key});
8
9   @override
10  State<ActividadesScreen> createState() => _ActividadesScreenState();
11 }
12
13 class _ActividadesScreenState extends State<ActividadesScreen> {
14   final DatabaseHelper _dbHelper = DatabaseHelper();
15   List<Actividad> _activities = [];
16   final Logger _logger = Logger();
17   final TextEditingController _nombreController = TextEditingController();
18
19   @override
20   void initState() {
21     super.initState();
22     _loadActivities();
23   }
24
25   Future<void> _loadActivities() async {
26     try {
27       final activities = await _dbHelper.getActividades();
28       setState(() {
29         _activities = activities;
30       });
31     } catch (e) {
32       _logger.e('Error al cargar actividades: $e');
33     }
34   }
35
36   Future<void> _addActivity() async {
37     if (_nombreController.text.isEmpty) return;
```